



COMP 4300

Intro to Game Programming

Lecture 20

C++ Visual Profiling

Singleton Classes

Singletons

- A Singleton is a programming **pattern**
- Typically implemented with OOP
- A Singleton class ensures that **only one instance** of the class can be created
- Typical use case is for 'global' functionality
 - System Logger, Blackboard, etc
- Implementation depends on language

C++ Singleton

- C++ makes **implementing** singleton easy
- We can make use of a **static** variable and a static function to ensure that only one instance of a class can be created
- We will also make the constructor of the class **private** so that no other code can 'accidentally' instantiate it

```
// Example C++ Singleton Class
1. class Singleton
2. {
3.     Singleton() { } // private constructor
4. public:
5.     static Singleton & Instance()
6.     {
7.         static Singleton instance; // only run once
8.         return instance;
9.     }
10.    void doThing() {}
11. };
```

```
// Outside usage: Singleton::Instance().doThing()
```

```
// File: Logger.hpp
1. class Logger
2. {
3.     std::ofstream m_fout = std::ofstream("log.txt");
4.     Logger() { }
5. public:
6.     static Logger & Instance()
7.     {
8.         static Logger instance;
9.         return instance;
10.    }
11.    void log(const std::string & str) { m_fout << str << "\n"; }
12. };
// Outside usage: Logger::Instance().log("Hello there");
```

Removing Instance() call

- If you get annoyed by the Instance() call, you can do some trickery to eliminate it
- Note: Is tedious if many public functions

```
// File: Timer.hpp
1. class Logger
2. {
3.     std::ofstream m_fout = std::ofstream("log.txt");
4.     Logger() { }
5.     static Logger & Instance()
6.     {
7.         static Logger instance;
8.         return instance;
9.     }
10.    void logImpl(std::string & str) { m_fout << str << "\n"; }
11. public:
12.    static void Log(std::string & str) { Instance().logImpl(str); }
13. }; // Outside usage: Logger::Log("Hello there");
```

Blackboards

- Game dev articles often refer to a data structure known as a 'BlackBoard'
- **Global** data storage / management
- Can be implemented as a Singleton
- Keeping everything in one class keeps it from polluting the global space
- Code is cleaner and safer than globals

Code Profiling

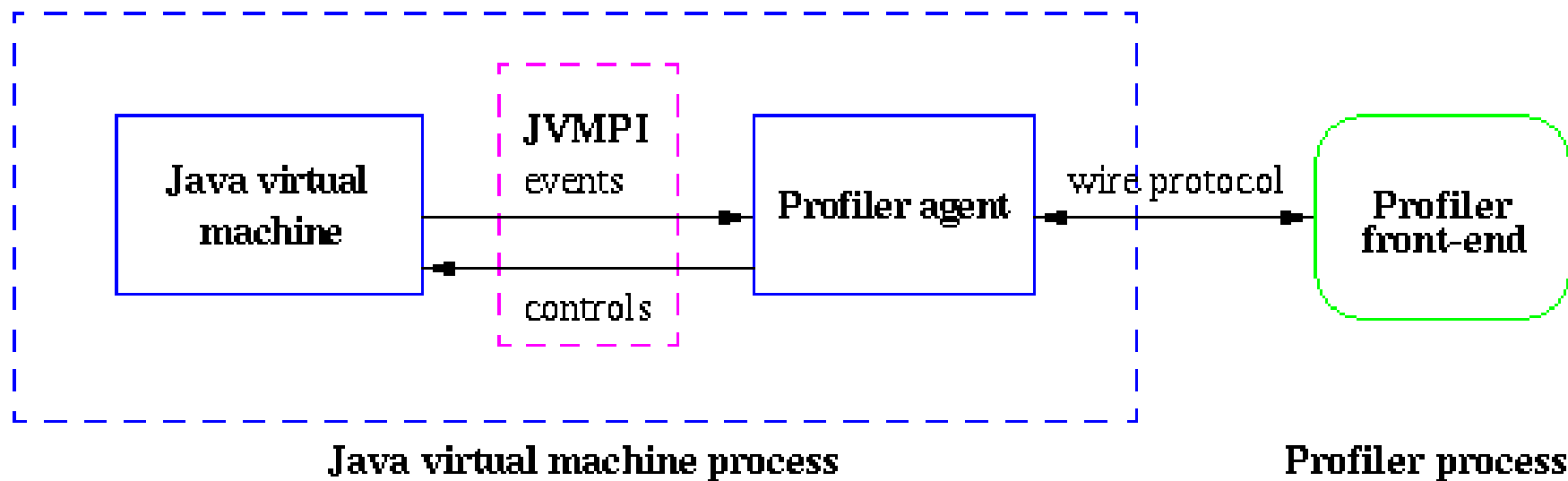
Profiling Code

- Profiling code involves timing specific functionality in order to see how long different functions/methods take to run
- Visual Studio has profiling tools built-in, which can help with profiling your code
- There are several types of code profilers

Event-Based Profilers

- Language or API has built-in events which trigger profiler data collection
- Provide hooks to events such as function calls, memory allocation, class construction, entering / leaving threads, etc
- Rely on specific functionality implemented in the language or API being used
- Lowest resolution of 3 main profiler types

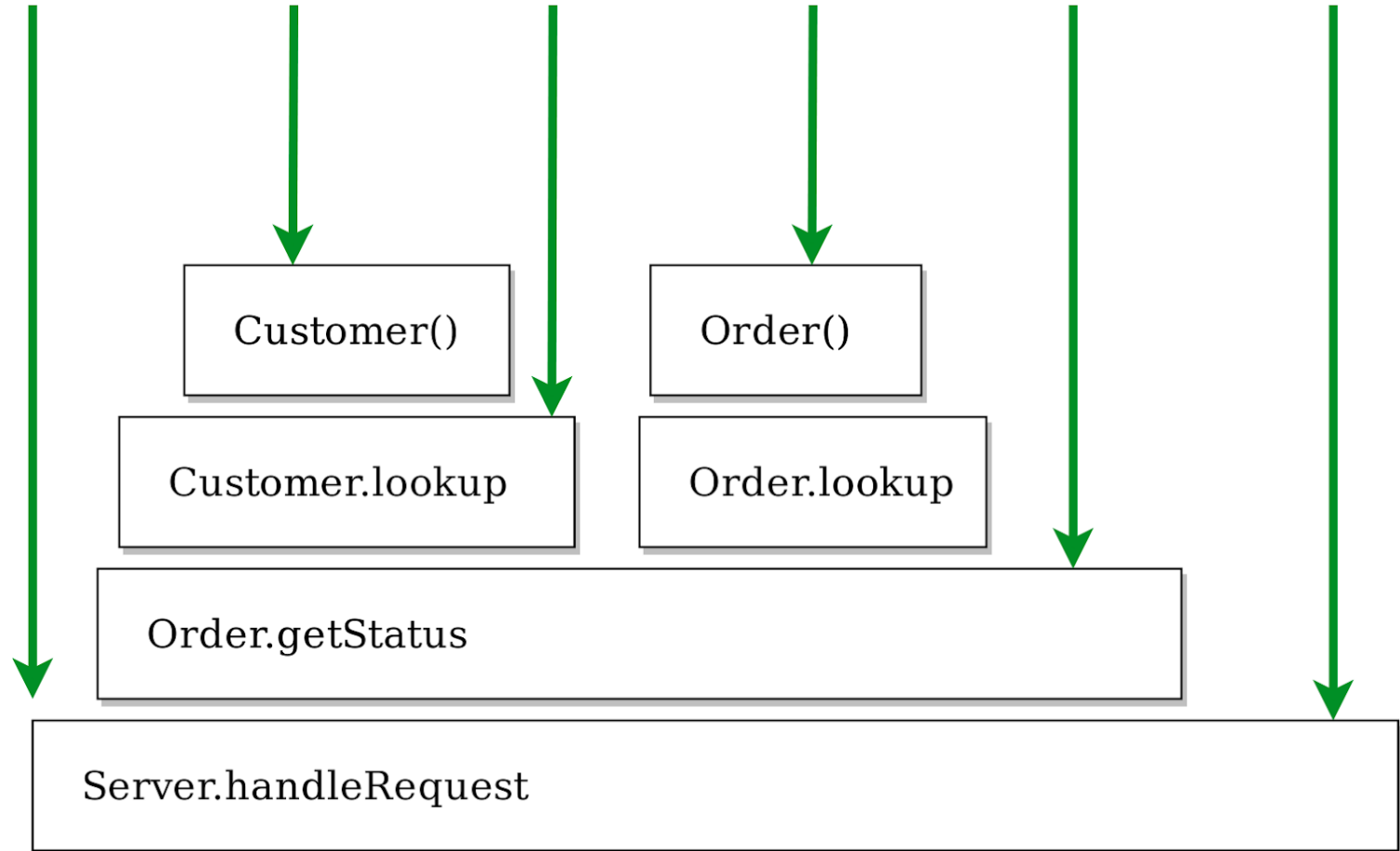
Event-Based Profilers



Statistical Profilers

- Collect profiling data via sampling
- The call-stack of a program is probed at regular intervals using OS interrupts
- Statistics about samples are displayed to the user for interpretation
- Example: 30% of samples collected showed the program was in the `render()` function
- Inexact results, but very little overhead

Sampling/Statistical Profiler



Instrumentation Profilers

- Instructions are added to the program to collect and output the desired type of profiling data
 - Can be done manually, by language, by compiler, etc
- Profiling data will be exact, however the introduction of instrumentation code can change the runtime of the program
- Starting / stopping timers, collating data, file output can all take tangible amount of time
- Allows fine control over what we want to profile
- **Let's implement a manual instrumentation timer**

```
1. int DoSomeCalculation()
2. {
3.     PROFILE_FUNCTION(); // Manual Instrumentation code insertion
4.     // some code here which takes a while
5. }
6. void MyFunction()
7. {
8.     PROFILE_FUNCTION();
9.     for (int i=0; i<10; i++) {
10.         PROFILE_SCOPE("LoopIteration"); // Profile 1 loop iteration
11.         ReadSomeData();
12.         DoSomeCalculation();
13.     }
14. }
```


DEMO

Visual Studio Profiler
Sampling / Instrumentation

C++ Code Timer

- Basic logic for timing code:
 - Record the starting time somewhere
 - Record the finish time somewhere
 - Elapsed = finish time – starting time
- `std::chrono` gives us timing tools
 - `std::chrono::high_resolution_clock::now();`
 - Can use this to calculate time elapsed

C++ Timer Class

- Having to manually record start and end times gets cumbersome, let's make a class
- C++ Timer class functionality
 - Store the start time via `start()`
 - Query how much time passed via `elapsed()`
 - Ability to start / stop the timer at will
 - Abstracts away the `std::chrono` syntax

```
// File: Timer.hpp
1.  #include <chrono>
2.  class Timer
3.  {
4.      long long m_start = 0, m_end = 0;
5.      std::chrono::time_point<std::chrono::high_resolution_clock> m_stp;
6.  public:
7.      Timer()      { start(); }
8.      void start() { m_stp = std::chrono::high_resolution_clock::now(); }
9.      long long elapsed() // returns time elapsed in microseconds
10.     {
11.         using namespace std::chrono;
12.         auto etp = high_resolution_clock::now();
13.         m_start  = time_point_cast<microseconds>(m_stp).time_since_epoch().count();
14.         m_end    = time_point_cast<microseconds>(etp).time_since_epoch().count();
15.         return m_end - m_start;
16.     }
17. };
```

```
// Example Timer usage
```

```
1. #include "Timer.hpp"
```

```
2. int main()
```

```
3. {
```

```
4.     Timer t;
```

```
5.     // maybe some other things here
```

```
6.     t.start();
```

```
7.     functionThatTakesSomeTime();
```

```
8.     auto elapsed = t.elapsed();
```

```
9.     std::cout << "Elapsed time is " << elapsed << "us\n";
```

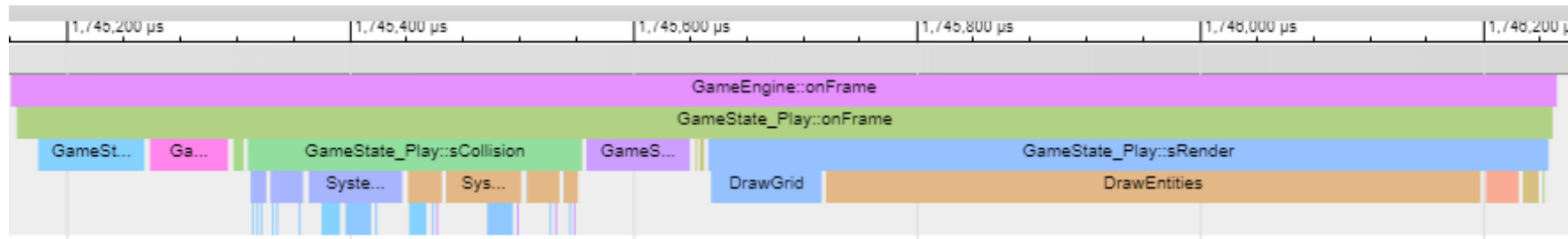
```
10. }
```

Visual C++ Profiling

- The following slides and code were inspired by a YouTube video by **TheCherno**:
 - “Visual Benchmarking in C++”
 - <https://www.youtube.com/watch?v=xIAH4dbMVnU>
- This code is an updated, simplified version with several bug fixes / improvements
 - TheCherno GitHub Discussion
 - <https://gist.github.com/TheCherno/31f135eea6ee729ab5f26a6908eb3a5e>

Visual Code Profiling

- Humans are visual creatures, staring at text output is difficult to understand
- What if we want to visually profile code?
- What if we could get something like this

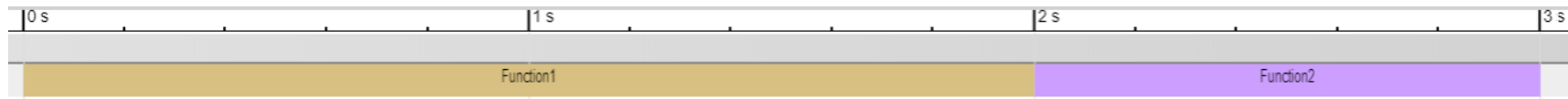


chrome://tracing

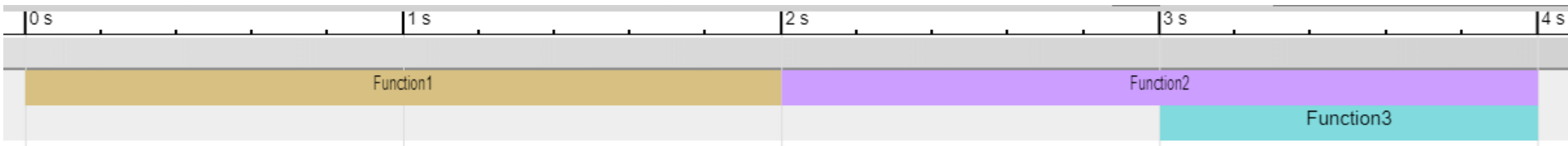
- It turns out that chrome has really nice visual tracing functionality built-in
- Initially meant for website profiling, it can input files in specific json format
 - [Chrome Tracing JSON Format Specification](#)
- If we can output our timing code with this specific format, chrome can visualize it!


```
// chrome://tracing json format
1. {
2.     "name": "myName",           // event name
3.     "cat": "category,list",    // event category
4.     "ph": "B",                 // event type (usually 'X')
5.     "ts": 12345,               // tracing clock timestamp (us)
6.     "pid": 123,                // process id
7.     "tid": 456,                // thread id
8.     "args": {                  // optional args for display
9.         "someArg": 1,
10.        "anotherArg": {
11.            "value": "my value"
12.        }
13.    }
```

```
// example tracing file
1. {
2.   "otherData": {},
3.   "traceEvents":
4.   [
5.     { "cat": "function", "dur": 2000565, "name": "Function1",
6.       "ph": "X", "pid": 0, "tid": 3414070038, "ts": 2515589687304},
7.     {"cat": "function", "dur": 1000526, "name": "Function2",
8.       "ph": "X", "pid": 0, "tid": 3414070038, "ts": 2515591688288}
9.   ]
10. }
```



```
1. {  
2.   "otherData": {},  
3.   "traceEvents":  
4.   [  
5.     {"cat": "function", "dur": 2000451, "name": "Function1",  
6.       "ph": "X", "pid": 0, "tid": 888960750, "ts": 2517249041745},  
7.     {"cat": "function", "dur": 1000327, "name": "Function3",  
8.       "ph": "X", "pid": 0, "tid": 888960750, "ts": 2517252043556},  
9.     {"cat": "function", "dur": 2001231, "name": "Function2",  
10.      "ph": "X", "pid": 0, "tid": 888960750, "ts": 2517251042698}  
11.   ]  
12. }
```



DEMO

Visual Profiling in C++

- How do we transition from the Timer class to outputting this JSON file to visualize?
- Timer class cannot output to file itself because all data needs to go in single file
- If only we had a way to globally write to a **single file** somehow...
- **Singleton** to the rescue!

```
// File: Profiler.hpp - Initial Implementation
```

```
1. class Profiler
2. {
3.     std::ofstream m_fout = std::ofstream("result.json");
4.     Profiler() { // write JSON header }
5. public:
6.     static Profiler & Instance()
7.     {
8.         static Profiler instance;
9.         return instance;
10.    }
11.    void writeProfile(const ProfileResult & r) { // write result JSON }
12.    ~Profiler() { // write JSON footer }
13. }; // singleton will destruct when program exits normally
// Main Idea: When Timer stops, call Profiler::Instance().writeProfile()
```

```
// File: ProfileResult struct
```

```
1. struct ProfileResult
```

```
2. {
```

```
3.     const std::string name = "Default";
```

```
4.     long long start        = 0;
```

```
5.     long long end          = 0;
```

```
6.     size_t threadID        = 0;
```

```
7. };
```

```
// Previous Timer elapsed() function pseudocode
```

```
1. long long Timer::elapsed()  
2. {  
3.     m_start = // calculate start time  
4.     m_end   = // calculate end time  
5.     return m_end - m_start;  
6. }
```

```
// new ProfileTimer stop() function
```

```
1. void ProfileTimer::stop()  
2. {  
3.     m_result.start = // calculate start time  
4.     m_result.end   = // calculate end time  
5.     Profiler::Instance().writeResult(m_result);  
6. }
```



```
// Example usage of new ProfileTimer
```

```
1. #include "ProfileTimer.hpp"
```

```
2. int main()
```

```
3. {
```

```
4.     ProfileTimer t("main");
```

```
5.     // maybe some other things here
```

```
6.     t.start();
```

```
7.     functionThatTakesSomeTime();
```

```
8.     t.stop(); // stops timer and Profiler logs it
```

```
9. }
```

Convenience is Key

- If we want to profile a lot of areas of code, manually starting and stopping the timer each time will be annoying, mess up code
- Let's develop a way to time a block of code by simply typing one command
- Any block of code that has this command will automatically be profiled to JSON

```
// Make ProfileTimer stop/log on destructor
1. int myFunc()
2. {
3.     ProfileTimer t("myFunc"); // timer starts on construction
4.     // some code runs here which takes a while
5. } // timer will stop and log on destructor call
6. void myFunction()
7. {
8.     for (int i=0; i<10; i++) {
9.         ProfileTimer t("LoopIteration"); // timer starts
10.        // do stuff in the loop
11.    } // ~Timer(), Profiler logs it
12. }
```

```
// File: Timer.hpp
1. #include <chrono>
2. class ProfileTimer
3. {
4.     ProfileResult m_result;
5.     bool m_stopped = false;
6.     std::chrono::time_point<std::chrono::high_resolution_clock> m_stp;
7. public:
8.     ProfileTimer(const std::string& name)
9.     {
10.         m_result.name = name;
11.         m_stp = std::chrono::high_resolution_clock::now()
12.     }
13.     ~ProfileTimer() { stop(); }
14. } // continued on next slide
```

```
// File: Timer.hpp
1. void ProfileTimer::stop()
2. {
3.     using namespace std::chrono;
4.
5.     if (m_stopped) { return; }
6.     m_stopped = true;
7.
8.     auto etp = high_resolution_clock::now();
9.     m_result.start = time_point_cast<microseconds>(m_stp).time_since_epoch().count();
10.    m_result.end    = time_point_cast<microseconds>(etp).time_since_epoch().count();
11.    m_result.threadID = std::hash<std::thread::id>{}(std::this_thread::get_id());
12.
13.    Profiler::Instance().writeProfile(m_result);
14. }
```

```
// Make ProfileTimer stop/log on destructor
1. int myFunc()
2. {
3.     ProfileTimer t("myFunc"); // timer starts on construction
4.     // some code runs here which takes a while
5. } // timer will stop and log on destructor call
6. void myFunction()
7. {
8.     for (int i=0; i<10; i++) {
9.         ProfileTimer t("LoopIteration"); // timer starts
10.        // do stuff in the loop
11.    } // ~Timer(), Profiler logs it
12. }
```

Convenience++

- This is still a little bit annoying - we don't want to have to manually name timers for every function we call it on
- Function names might change, then we have to go edit the timers too
- What if we want to turn profiling on/off?
- Let's create a **macro** we can call that does this all for us automatically

```
// C++ Macro for profiling calls
```

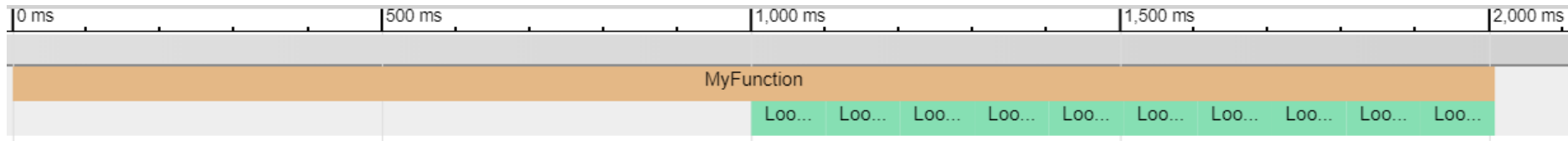
```
1. #define PROFILING 1
2. #ifdef PROFILING
3.     #define PROFILE_SCOPE(name) \
4.         ProfileTimer timer##__LINE__(name)
5.     #define PROFILE_FUNCTION() \
6.         PROFILE_SCOPE(__FUNCTION__)
7. #else
8.     #define PROFILE_SCOPE(name)
9.     #define PROFILE_FUNCTION ()
10. #endif
```


Macro #ifdef

- We define a macro: PROFILING
- If PROFILING is **defined**, then the profiling macro is defined normally, causing the program to log to the JSON file
- If PROFILING is **not defined**, then the macro is left blank by the preprocessor, turning off profiling with 0 overhead

```
// Using the ProfileTimer macros
1. int myFunc()
2. {
3.     PROFILE_FUNCTION();
4.     // some code runs here which takes a while
5. }
6. void myFunction()
7. {
8.     for (int i=0; i<10; i++) {
9.         PROFILE_SCOPE("LoopIteration");
10.        // do stuff in the loop
11.    }
12. }
```

```
// Using the ProfileTimer macros
1. void MyFunction()
2. {
3.     PROFILE_FUNCTION();
4.     std::this_thread::sleep_for(1s);
5.     for (int i=0; i<10; i++)
6.     {
7.         PROFILE_SCOPE("LoopIteration");
8.         std::this_thread::sleep_for(100ms);
9.     }
10. }
```



DEMO

`/code/` folder on course website